When Does Automated Alignment Search Find a Causal Mechanism?

Experiments on Carry Propagation and Graph Successor Prediction

Surya V

October 23, 2026

Abstract

Causal abstraction asks whether a neural network can be described by a simpler, human-readable causal model. Automated alignment search tries to find the internal activations that should be exchanged when we intervene on a variable in that high-level model. In this work I test this idea on two tasks: ones-column carry in addition using Qwen3-1.7B, and successor prediction in a small Transformer trained on shuffled graph edges. I compare coordinate-level alignment search with a DAS-style method that learns directions inside residual activations. The results are mixed. In the graph task, vanilla search repeatedly selects the final query-token site and gets 100% held-out interchange accuracy. However, other sites at the same token are almost as sufficient, and full activation patching of several random sites can also work well. DAS selects three query-token sites, but its learned projected directions reach only about 56.5% held-out IIA. In the carry task, search finds candidate sites, but the results do not support a strong causal claim: performance drops on held-out data, reverse interventions are imperfect, and same-carry controls are damaged. The experiments show why high interchange accuracy is not enough by itself. Search can find a useful place to patch without identifying the computation that produced the information, and the high-level causal model still has to be supplied by the researcher.

1 Introduction

The goal of mechanistic interpretability is not only to predict what a model will output. We also want to understand the computation that produces the output. Causal abstraction gives one way to state this goal precisely: write down a high-level causal model, then test whether selected internal variables of a neural network behave like the variables in that model under interventions [1, 2, 3].

The practical problem is the alignment. Suppose a high-level model says that a variable such as a carry bit or a successor node determines the answer. We need to find which low-level activations in the Transformer correspond to that variable. Automated alignment search optimizes a mask over candidate activations. DAS extends this idea by learning a direction or subspace in the activation space, instead of requiring the high-level variable to align with whole coordinates [2].

I wanted to test a fairly direct question: when the search finds an alignment that works under an interchange intervention, does it actually identify the mechanism we intended? To test this, I used two tasks with different levels of control. The carry task uses a pretrained language model and a manually specified arithmetic variable. The graph task uses a small Transformer trained from scratch, where the task and the candidate sites are easier to inspect.

The results are useful, but they are not a final demonstration that automated alignment search can discover causal models on its own. In fact, the main limitation that appears throughout the

experiments is that the search objective only knows the high-level variable and counterfactual targets that we give it. It does not discover the high-level causal model automatically.

2 Mathematical setup

2.1 High-level and low-level variables

Let N be the neural network and let H be a high-level causal model. The high-level model contains a variable V, such as a carry flag or the successor of a node. Let $\mathcal{C}$ be the set of candidate low-level sites. A site may be specified by a layer and token position, for example (ℓ, p).

The alignment search tries to find a subset $S \subseteq \mathcal{C}$ whose activations can be exchanged between two examples to reproduce the counterfactual predicted by H. Write B for the base example and X for the source example. Their activations at a candidate site i are h_i^B and h_i^X.

For coordinate-level patching, define a binary mask $m_i \in \{0, 1\}$. The intervened activation is

$$h_i^{\text{int}} = (1 - m_i)h_i^B + m_i h_i^X = h_i^B + m_i(h_i^X - h_i^B). \tag{1}$$

If $m_i = 1$, the source activation replaces the base activation at that site. If $m_i = 0$, the base activation is left unchanged. The network then continues its forward pass using the intervened activation.

Let $y^*(B, X)$ be the correct high-level counterfactual answer. This is not necessarily just the source answer in every possible task; it is the answer predicted by the high-level model when the relevant high-level variable is set to its source value while the rest of the base example is retained. The interchange-intervention accuracy is

$$\text{IIA}(S) = \frac{1}{n} \sum_{j=1}^{n} \mathbf{1}\left[\hat{y}_N(B_j \leftarrow X_j; S) = y^*(B_j, X_j)\right]. \tag{2}$$

In these experiments the target is a single answer token, so IIA is the fraction of pairs for which the patched base prediction equals the expected counterfactual answer.

2.2 The optimization objective

The alignment is represented by trainable mask logits α_i. A differentiable relaxation produces soft inclusion probabilities, while a straight-through or thresholded mask is used for the forward intervention. The objective has two terms:

$$\mathcal{L}(\alpha) = \mathbb{E}_{(B,X)\sim\mathcal{D}}\left[-\log p_N\big(y^*(B, X) \mid B \leftarrow X; m(\alpha)\big)\right] + \lambda \sum_{i\in\mathcal{C}} f(\alpha_i). \tag{3}$$

The first term encourages the intervention to produce the desired counterfactual. The second penalizes selecting too many sites. The parameter λ controls the sparsity–accuracy trade-off. If it is too large, the selected mask can become empty; if it is too small, the search can select more sites than necessary.

For the carry task, the implementation uses a sigmoid-based binary-concrete mask with a straight-through hard forward pass. In the graph task, vanilla search uses a sparse mask over candidate sites. These are related objectives, but the implementation details and hyperparameters are not identical.

2.3 Vanilla patching and DAS

Vanilla alignment search exchanges the full activation vector at a selected site. This is a broad intervention: the vector may contain the variable of interest, but it also contains other information.

DAS instead learns a direction v_i at each selected site. For a unit direction, the projected intervention is

$$h_i^{\mathrm{DAS}} = h_i^B + \left((h_i^X - h_i^B)^\top v_i\right) v_i. \tag{4}$$

Only the component of the source–base difference along v_i is transferred. This is a more restrictive intervention than full replacement. It is also why full-patch IIA and projected-DAS IIA must be reported separately: they evaluate different interventions.

3 Experiment 1: ones-column carry in a pretrained language model

3.1 Task and candidate sites

The first task uses Qwen3-1.7B as a next-token predictor. Prompts have the form

$$a + b = \text{hundreds digit},$$

and the model is evaluated on the next token, the tens digit of the sum. Each matched base/source pair keeps the hundreds and tens digits of the addends fixed and changes only the ones digits. The base example has no carry from the ones column, while the source example has a carry. The examples are constructed so that the source tens digit is one greater than the base tens digit and no carry from the tens column changes the hundreds digit.

The purpose of this experiment is to test whether automated search can locate activations that causally carry the ones-column carry into the tens-digit prediction, rather than merely finding activations correlated with the answer. The high-level variable is therefore the ones-column carry:

$$C_{\mathrm{ones}} = \mathbf{1}[a_{\mathrm{ones}} + b_{\mathrm{ones}} \geq 10].$$

The intended intervention transfers the carry-relevant information from a source with a carry into a base example without a carry. The expected answer is the source tens digit.

The first coordinate-level search used 300 screened matched pairs and 84 candidate layer–position sites at token positions 7, 8, and 9. We deliberately restricted the search to layers 0–15, rather than all layers in the model (`model.config.n_layers`). In preliminary unrestricted searches, optimization tended to select late-layer sites, including layers 24 and 27, at the final token, where the answer-relevant information was already directly available. Such a site can make the counterfactual answer appear by transferring a downstream representation, without locating where the carry computation is performed. The layer restriction was therefore an attempt to search earlier in the computation; it is a researcher-imposed prior and does not guarantee that the selected sites implement the carry mechanism. Screening was important because the model had to predict both clean base and source answers correctly before an example was used for search. On the filtered set, both base and source accuracy were 100%. Without an intervention, the model never produced the source tens digit on these carry-changing pairs, so the empty-mask IIA was 0%.

3.2 Coordinate-level search results

In the saved ones-carry run, the search selected four sites from 96 candidates:

$$\{(3, 7), (14, 8), (15, 9), (15, 10)\}.$$

On the 300 screened pairs used for optimization, IIA was 51.7%: the patched model predicted the source tens digit in about half the cases. The mean for 50 random masks of four sites was 32.9%, so the search did better than this baseline, but its result remained modest and was measured on the optimization data, not a held-out set.

The leave-one-out ablation asks whether each selected site is necessary for the discovered intervention: remove one site while keeping the other selected sites, then recompute IIA. Removing $(3, 7)$ reduced IIA from 51.7% to 0%; removing $(15, 9)$ or $(15, 10)$ left it at 51.7%, and removing $(14, 8)$ gave 52.0%. Thus, in this run, $(3, 7)$ was uniquely critical to the intervention's observed effect, while the other three sites added little measurable benefit. This is an interesting causal-localization clue, but not proof that $(3, 7)$ represents the carry variable: I did not inspect its activation content, and a leave-one-out score alone cannot tell us what information it contains or whether the model computes the carry there. Follow-up work should inspect and decode the activation at $(3, 7)$ and test it with held-out examples, reverse interventions, and same-carry controls.

3.3 DAS-style subspace search and validation

I next used a faster batched DAS implementation and explicitly evaluated learned subspaces. The candidate set contained 96 sites across layers 0–15 and positions 5–10. The screened development data had 300 matched pairs; in a separate unfiltered pool, 354 of 384 pairs had both base and source answers correct.

With subspace dimension 8, one run selected site $(6, 7)$. **IIA** (interchange intervention accuracy) is the percentage of examples where swapping the selected activation information makes the model give the counterfactual answer. For example, if the base addition has no ones-column carry and the source addition does, a successful forward intervention makes the base predict the source's tens digit.

The initial search IIA was 73.0% on the search examples. We then **refined** the subspace: we kept site $(6, 7)$ fixed and continued training its direction on the training examples. Training IIA rose to 81.0%, but **held-out IIA**—the same test on examples not used to fit the subspace—was only 50.7%. This drop suggests the refined direction did not generalize well.

The **reversed intervention** tests the opposite direction: transferring information from a carry example into a no-carry example, with the expected answer changed accordingly. Its IIA was 49.0%. The **same-carry controls** test cases where both examples have the same carry status, so the answer should stay the same. Accuracy was 82.0% for no-carry to no-carry and 86.5% for carry to carry. These drops show that the learned subspace sometimes changes the answer even when the carry status should be unchanged. Together, the results do not establish a clean carry representation.

A higher-dimensional run with subspace dimension 64 selected three sites, $(1, 8)$, $(2, 8)$, and $(10, 9)$. Its refined training IIA was 83.7%, but held-out IIA was 43.0%, and the reversed test was close to zero. This gap is consistent with overfitting and shows that higher training IIA does not necessarily give a more faithful causal abstraction.

The notebook also contains a locked evaluation for the frozen rank-8 candidate at $(6, 7)$. It uses 300 forward pairs, 300 reversed pairs, and 200 examples in each same-carry control. On the forward task, the learned subspace achieved 53.3% accuracy, compared with 0% without

intervention and 54.0% for full residual replacement. The paired-bootstrap confidence interval for the learned subspace minus no intervention was +0.477 to +0.590. However, the learned subspace was not significantly better than full residual replacement: the difference was −0.007, with a 95% interval from −0.057 to +0.043.

The same candidate reached 45.7% on the reversed task, while full replacement reached 67.0%. On the unchanged no-carry control, the learned subspace was correct on 78.5% of examples, compared with 100% without intervention. On the unchanged carry control, it reached 90.5%, again below the 100% no-intervention score. So the intervention does affect carry-related behavior, but it does not isolate a clean carry variable. The control failures are important: a good causal abstraction should not change outputs when the high-level variable is meant to remain unchanged.

Finally, the negative-control search restricted to positions 5 and 6 selected no nodes. This is a failed discovery under the current optimization settings. It does not prove that the relevant information is absent from those positions; it only says that this run did not find a non-empty mask.

4 Experiment 2: graph successor prediction in a small Transformer

4.1 Task

The second task is more controlled. Each input contains four directed edges forming a path through the nodes A, B, C, D, E. The edges are shuffled, then a query node is given. The model must output the node immediately following the query in the underlying path.

For example, for the path

$$C \to A \to E \to D \to B,$$

the answer to query E is D. The Transformer was trained from scratch on 20,000 examples, with 2,400 validation and 2,400 test examples. The intervention analysis uses matched base/source pairs with the same graph and edge order but different query nodes and different correct answers. We keep pairs only when both clean predictions are correct. There are 500 search pairs and 300 held-out pairs.

The candidate set is deliberately small: six residual-stream sites at layers $0, 1, 2$ and token positions 8 and 9. Position 8 is the query separator Q; position 9 contains the query node. This restricted domain makes the search easier to inspect, but it also limits the conclusions: we are not searching all tokens or all layers of the network.

4.2 Initial vanilla and DAS results

Vanilla search selected the single site $(2, 9)$ and obtained 100% training and held-out IIA. Its one-site random-patch baseline averaged 43.1% with a standard deviation of 49.3 percentage points. DAS selected the three query-token sites $(0, 9), (1, 9), (2, 9)$. Full activation replacement at these sites also reached 100% held-out IIA. The random three-site full-patch baseline averaged 89.6% with a standard deviation of 29.9 percentage points.

The learned DAS directions told a different story. The projected intervention achieved only 56.0% held-out IIA in the initial run. This distinction is central: the selected sites can work when their full activations are copied, but the learned rank-1 directions do not transfer the needed information reliably.

4.3 Ablation of query-token sites

We tested each query-token site alone, every pair, and all three sites together. The results are shown in Figure **??**. Patching $(0, 9)$ alone achieved 99.0%, $(1, 9)$ achieved 99.33%, and $(2, 9)$ achieved 100%. All pairs and the three-site combination were between 99.67% and 100%.

This is a strong sufficiency result but a weak uniqueness result. Each of the three sites can independently support the desired intervention when its full activation is replaced. The random full-patch baseline was already 59.8% on average for one site, 79.6% for two sites, and 92.9% for three sites. As the number of patched sites increases, random full activation replacement becomes a strong baseline. This means that the 100% score for three selected sites cannot, by itself, be interpreted as discovery of the internal algorithm.

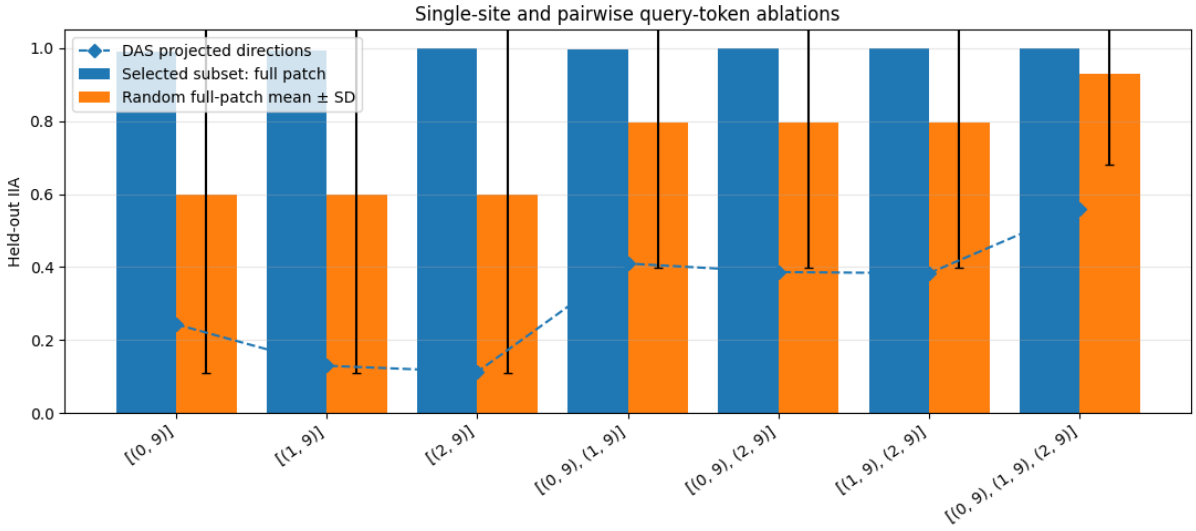


Figure 1: Full activation-patching ablations of the query-token sites, with random full-patch baselines. The dashed series shows the projected DAS scores available in the notebook. Full patching and projected DAS are different interventions.

4.4 Iterative exclusion

To check whether the search was merely finding the same convenient site repeatedly, I excluded the selected sites and reran the search on the remaining candidate set. Vanilla first selected $(2, 9)$, with 100% held-out IIA. After excluding it, the next search selected $(0, 9)$ and $(1, 9)$ and retained 99.67% IIA. When all query-token sites were excluded, the remaining separator sites at position 8 produced 0% IIA.

DAS selected all three query-token sites in its first round. Full-patch IIA was 100%, while projected DAS IIA was 58.7%. After excluding the query-token sites, the remaining separator-site searches produced 0% IIA under both full patching and projected evaluation. Figures 2–3 show the sequential results.

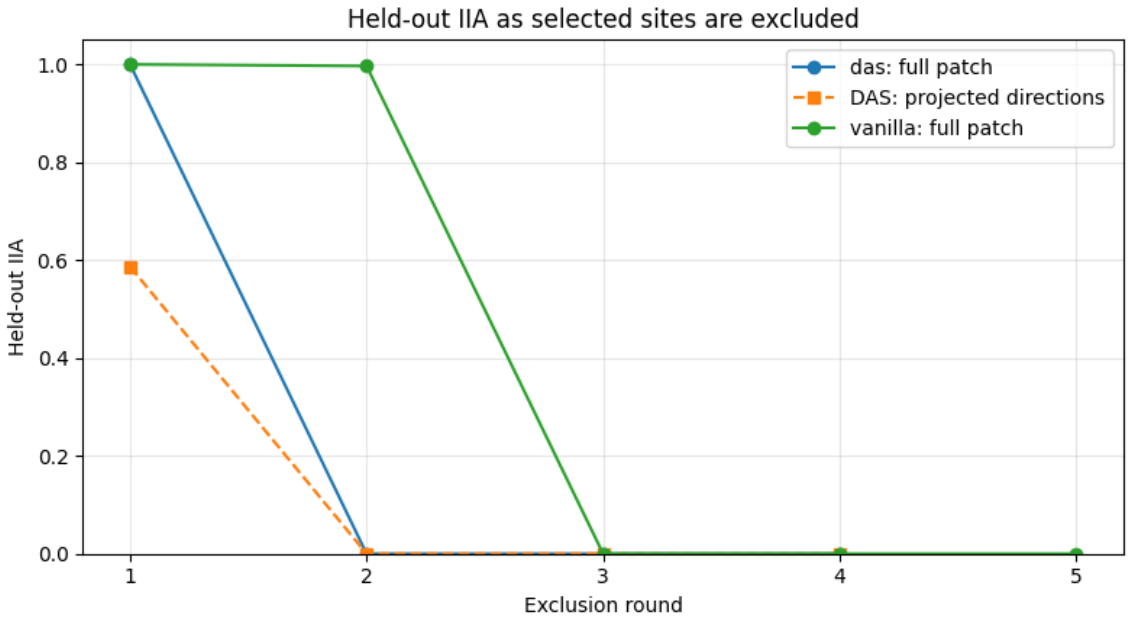


Figure 2: Held-out IIA as sites selected in earlier rounds are excluded. The candidate set contains only six sites, so the experiment ends once all candidates have been removed.

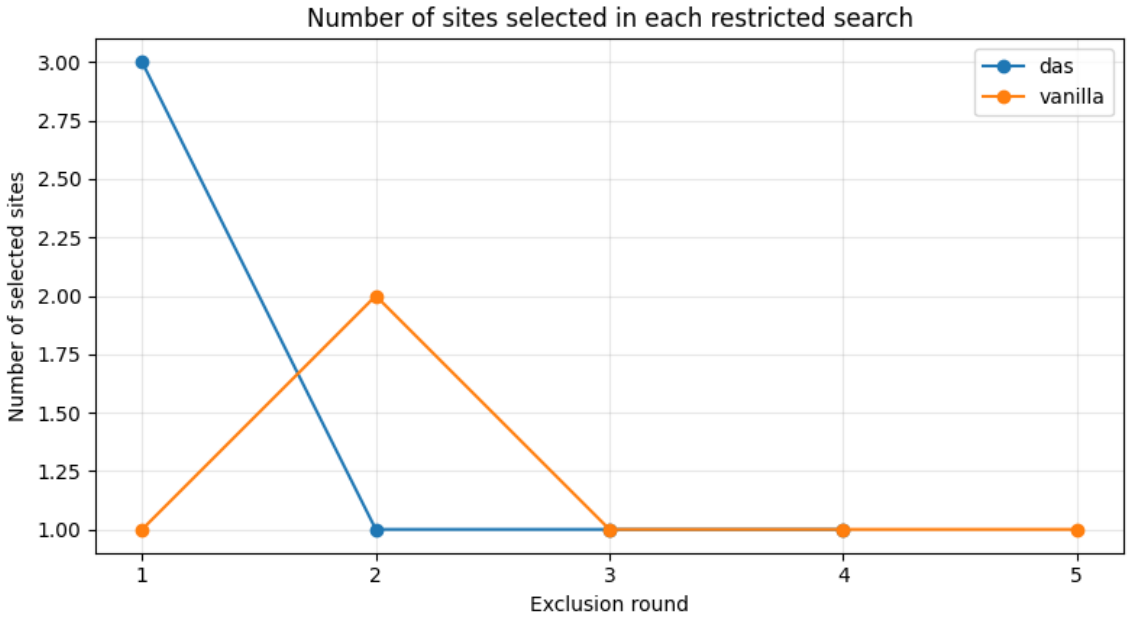


Figure 3: Number of sites selected in each restricted search round.

The result suggests that, within this candidate set, query-token sites are much more useful for the tested interchange behavior than the separator sites. It does not show that the separator contains no relevant information, because the search space and intervention operator are limited.

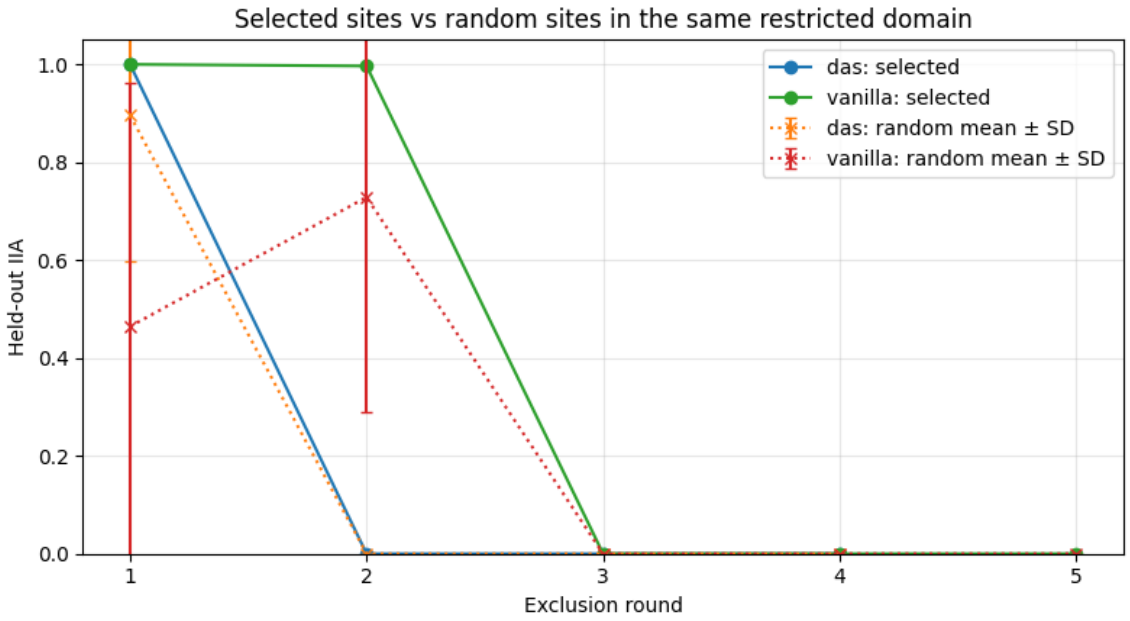


Figure 4: Selected-site IIA compared with random-site controls drawn from the restricted domain available in each round.

4.5 Multi-seed stability

I repeated vanilla and DAS search with five optimization seeds while keeping the trained Transformer and data fixed. Vanilla selected $(2, 9)$ in all five runs and reached 100% held-out IIA each time. DAS selected $(0, 9), (1, 9), (2, 9)$ in all five runs. Full-patch IIA was 100% in every run, but projected DAS IIA ranged from 55.0% to 58.0%, with a mean of 56.47% and standard deviation of about 1.07 percentage points.

The site-selection frequency and IIA results are shown in Figures 5 and 6. This gives evidence that the optimization is stable for this particular trained model and restricted candidate set. It does not establish stability across independently trained Transformers.

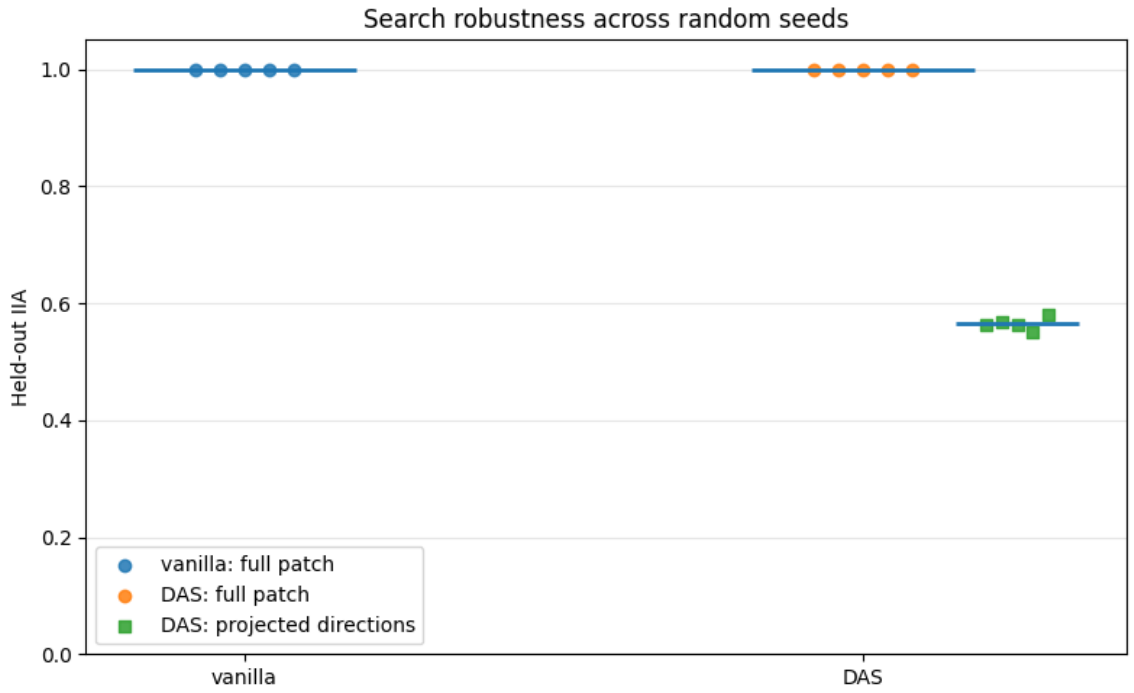


Figure 5: Held-out IIA across five search seeds. DAS full-patch IIA is separated from projected-direction IIA.

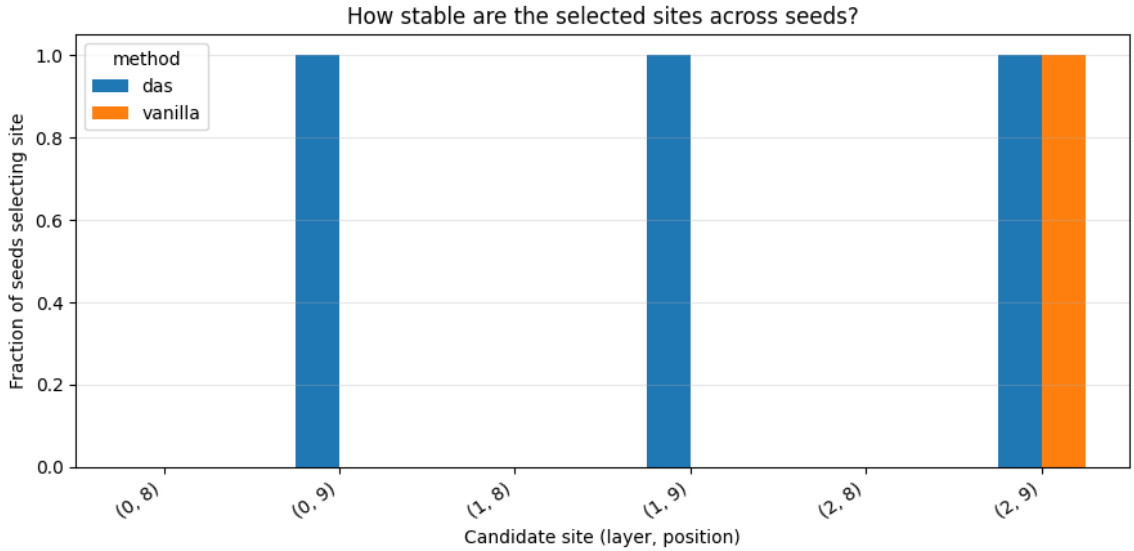


Figure 6: Fraction of search seeds in which each candidate site was selected. The same model was used across seeds.

5 Discussion

5.1 A site that works is not necessarily the site where the computation happens

The main issue in these experiments is the difference between finding an activation that can carry a counterfactual and finding the internal computation that produced it.

In the graph task, the search is allowed to consider the final query-token residual stream after several layers of computation. By that point, the answer-relevant information may already be available. Copying the entire residual vector from the source example can therefore make the base example produce the source answer. The search can succeed by selecting a downstream location where the result is already present, rather than finding the earlier operation that computed the successor.

This is not necessarily a bug in the optimization. It follows from the objective: the objective rewards counterfactual output agreement and sparsity, not the discovery of an earliest computation site. Restricting candidate sites to earlier layers or particular tokens can help answer a more specific question, but the restriction itself is a researcher-supplied assumption. It may exclude the actual mechanism or bias the result toward the locations we expected.

The graph experiment makes this point especially clearly. The query-token sites work well, but all three are nearly sufficient under full activation replacement. The method has found locations that support the intervention; it has not yet distinguished whether these locations compute, store, relay, or merely expose the relevant information.

5.2 The high-level causal model is still supplied by us

The second limitation is more fundamental. In both tasks, we already specified the variable we wanted to find. For addition, it was the ones-column carry. For graph successor prediction, it was the high-level relation between the query node and its successor. We also constructed matched examples and provided the counterfactual target for each pair.

The search does not discover these variables or infer the causal graph. It solves the alignment problem conditional on the high-level model, candidate sites, data pairs, and target labels that we provide. This limitation is related to a broader concern raised by Data-driven Circuit Discovery: hypothesis-driven methods can impose a single task/circuit view even when different examples may be processed by different mechanisms, and discovered circuits can change under semantically minor dataset changes (Rai et al., 2026). This paper is a useful motivation, not evidence that it explains the carry results specifically. If the high-level model is incomplete or wrong, a high IIA score may reflect a partial or unintended alignment. There is currently no automatic procedure in these experiments that starts from raw model behavior and discovers the right higher-level causal variables and their dependencies.

5.3 IIA is not enough

Several results show why IIA must be interpreted with controls.

First, full activation replacement is a high-bandwidth intervention. It transfers the entire activation vector, not just the variable of interest. Second, multiple sites can be sufficient, as shown by the graph-task ablations. Third, random multi-site patches can perform surprisingly well. Fourth, the carry-task candidate can improve the forward counterfactual while damaging the same-carry controls. A method can therefore obtain non-trivial forward IIA without implementing the intended high-level variable faithfully.

For DAS, another issue is that the learned projected directions are not equivalent to full activation patches. In the graph task, full-patch IIA is 100%, while projected IIA is about 56.5%. Reporting only the full-patch score would hide the main failure of the learned subspace. A fair comparison requires controls that use the same intervention operator, including random directions at the same sites.

5.4 Other limitations

There are several further limitations in the current experiments.

- **Restricted candidate sets.** The graph experiment searches only six sites. The carry experiments use manually chosen token positions. Results are conditional on these choices and cannot establish that other layers or positions are irrelevant.
- **Data screening.** Carry pairs were filtered so that the model answered both base and source prompts correctly. This is necessary for clean intervention tests, but it also narrows the input distribution and can introduce selection effects.
- **Limited generalization evidence.** The first carry search reports IIA on the same pairs used for optimization. The later DAS study has held-out and locked tests, but those results are only around chance for the forward task and are accompanied by control failures.
- **Optimization and threshold sensitivity.** Different sparsity strengths and subspace dimensions select different numbers of sites. Some settings select no nodes. An empty mask is a failed discovery under those settings, not evidence that the variable is absent.

6 Conclusion

The experiments give a simple answer to the question I started with. Automated alignment search can find sites that support an intended interchange intervention, and in the graph task vanilla search does this consistently. But this is not the same as automatically discovering the model's internal algorithm.

The carry task is a cautionary result: the search finds candidate sites, but held-out performance and controls do not support a clean carry abstraction. The graph task is easier to interpret, yet its very high full-patch scores are compatible with redundant answer information at several query-token sites. DAS is particularly revealing because the selected sites work under full replacement while the learned projected directions do not reproduce that result.

For the next step, I would focus on matched intervention controls, broader candidate sets, and tests that distinguish early computation from late availability of the answer. More fundamentally, an end-to-end discovery method would need to propose the high-level causal variables and their graph, not only find an alignment for a variable that we have already specified.

References

[1] A. Geiger, H. Lu, T. Icard, and C. Potts. Causal Abstractions of Neural Networks. In *Advances in Neural Information Processing Systems*, 2021. https://arxiv.org/abs/2106.02997.

[2] A. Geiger, Z. Wu, C. Potts, T. Icard, and N. Goodman. Finding Alignments Between Interpretable Causal Variables and Distributed Neural Representations. In *Proceedings of the Third Conference on Causal Learning and Reasoning*, PMLR 236, pp. 160–187, 2024. https://proceedings.mlr.press/v236/geiger24a.html.

[3] A. Geiger et al. Causal Abstraction: A Theoretical Foundation for Mechanistic Interpretability. *Journal of Machine Learning Research*, 26(83):1–64, 2025. https://www.jmlr.org/papers/v26/23-0058.html.

[4] D. Rai, M. Geva, and Z. Yao. Data-driven Circuit Discovery for Interpretability of Language Models. arXiv:2605.09129, 2026. https://arxiv.org/abs/2605.09129.